

COURSE

Python for Business with Copilot

Jacob Coles

Data scientist



Day 1 · Working with Copilot, controls and reporting automation

Today

Start	Session	Topic
09:00	D1-S01	Introduction and choosing the right tool
09:25	D1-S02	Environment verification
09:45	D1-S03	Working with Microsoft 365 Copilot
10:45	D1-S04	The working method end to end
13:15	D1-S05	Python in Excel
14:00	D1-S06	Data quality, controls and reconciliation
15:00	D1-S07	Reporting automation
15:45	D1-S08	Guided end-to-end exercise

Breaks 10:30–10:45 and 14:45–15:00 · Lunch 12:15–13:15

D1-S01 · 09:00

Introduction and choosing the right tool

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

What this course is for

You will learn to

- **Describe** a business requirement precisely
- **Generate** Python with Microsoft 365 Copilot
- **Run** it in VS Code or Excel
- **Check** it against figures you already trust
- **Correct** it and **hand it over**

Four days

Day	Focus
1	Copilot, controls, reporting automation
2	External data, planning, integration
3–4	Your own scoped project

KEY IDEA

Success is not “the code ran”. Success is **a checked result that someone else can rerun.**

Meet Northbridge Business Services

A **fictional** services company with three units:

Unit	Name
N01	Support Services
N02	Training Services
N03	Advisory Services

You are the operations analyst. Every month each unit sends a transaction export (a CSV file).

Why month-end goes wrong today

- Files are **copy-pasted** into one workbook; nobody knows what was pasted twice
- Account-code lookups **disagree**; `0400` silently becomes `400`
- A unit **forgets to send** its file; the total just looks a bit low
- The same transaction number exists in **several units**
- **No trail** of what was checked

Where we are going: the finished December pack

The **December pack** is what management receives at month-end:

1. A formatted workbook with five sheets:

Sheet	What it shows
Summary	Each unit: revenue, costs, operating result vs budget
ByLine	The 9 reporting lines behind it
Controls	The nine checks and their status
Exceptions	Any failed records, with file and row
ReadMe	Inputs used and how to rerun

2. Evidence files: `control_summary.csv`, `exceptions.csv`

3. A handover note for a colleague

December at a glance

Operating result **67,330.86**

Budget 67,600.00 · Variance –269.14

Controls: **9 of 9 pass** · Release: **ready**

3 unit files



Controls



Workbook



Handover

By **17:00** today you will have built a pack like this yourself, for January.

Reading the figures: eight rows from N01

Line	Report line	Amount	Status
T001	revenue	1000.00	posted
T002	revenue	500.00	posted
T003	revenue	-100.00	posted
T004	direct_cost	-400.00	posted
T005	direct_cost	-200.00	posted
T006	direct_cost	+20.00	posted
T007	overhead	-100.00	posted
T008	revenue	999.00	cancelled

Three rules

1. Revenue is stored **+**, costs are stored **-**. So the result is a plain **sum**.
2. Only **posted** lines count. Cancelled lines stay in the file but are left out.
3. A **+** on a cost line is a **reversal** (a cost undone). Keep its sign.

CHECK

Seven posted lines added as stored: **720.00**

Three plausible answers, one right one

720.00

Right: posted lines, signs kept

1,719.00

Wrong: the cancelled T008
(999.00) was counted

680.00

Wrong: costs made positive first,
so the +20 reversal became extra
cost

KEY IDEA

All three come from code that **runs without an error**. The difference is a **business rule**, not Python. That is why every exercise today starts with a number you expect.

The working loop you will use all week



Microsoft 365 Copilot Chat

- Suggests code from your description
- Sees **only what you paste or upload**, not your folders
- Does **not** run code on **your machine** or in VS Code

You

- Run the code on your machine
- Compare the result with the expected figure
- Decide whether it can be used

Choosing the right tool

Ask four questions, in order: **task fit** · **maintenance** · **data access** · **handover**

Tool	Good at	Watch out for
Excel / PivotTable	Quick summaries of a table already open	Manual steps are hard to repeat
Power Query	Repeatable import; combining a folder of same-shaped files	Business checks must be added deliberately
Python in Excel	Analysis and charts with pandas inside a workbook	Cloud only: cannot read local files or the internet
Local Python (VS Code)	Repeatable multi-step processes: controls, a dated pack	Needs a Python setup and someone who can read code
Power BI / KNIME	Shared dashboards / visual workflows	Python inside them needs its own setup

KEY IDEA

Several files **alone** do not justify Python. Controls + exceptions + release decision + formatted handover do.

Two "Python + Excel" routes: never confuse them

Python in Excel

- Code lives **in a cell** (`=PY( . . . )`)
- Runs in **Microsoft's cloud**
- Reads **only the workbook**
- Today at **13:15** (D1-S05)

Local Python writing Excel

- Code lives in a **script or notebook**
- Runs on **your machine**
- Reads **files**, writes a **new .xlsx**
- Today at **15:00** (D1-S07)

Your turn: pick a tool and one reason

EXERCISE

Open sessions/D1-S01/learner/scenarios.md · **In pairs** · **5 min**

For each scenario, agree on **one tool** you would recommend and **the deciding reason**. A reason about maintenance or data access is stronger than “I prefer it”.

	Situation
A	One sales table a month, already in a workbook; ten minutes; nobody else runs it
B	Twelve regional CSVs; completeness, duplicate, mapping and control-total checks; a formatted pack; a colleague must rerun it
C	A Power BI report maintained by the BI team needs one baseline calculation; the analyst knows Python, the BI team doesn't

Next: prove your environment works

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

You now know

- What the December pack contains
- Why **720.00**, not 1,719.00 or 680.00
- How to choose a tool and justify it

Next (09:25, D1-S02)

Open the course folder in VS Code, select the course Python, and run the readiness check. Everyone ends **READY** or with a recorded plan.

D1-S02 · 09:25

Environment verification

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

Prove it, don't assume it

Before we touch Northbridge's figures, we check the tools that will produce them.

This is **the first control of the day**: a claim ("my setup works") needs evidence.

KEY IDEA

By 09:45 everyone is **READY**, or has a **recorded plan** (a fix, or working with a partner).

Five steps (Steps 1 to 5 in the participant guide)

Open the folder



Choose Python



Run the check



Find your files



Report

Guide: `sessions/D1-S02/participant_guide.md`

Setup notes: `sessions/D1-S02/learner/orientation_worksheet.md`

Step 1: Open the course folder itself (3 min)

VS Code → **File** → **Open Folder...** and choose the folder that **contains** these:

course folder/	<- open THIS one
case_pack/	data, templates
sessions/	one per session
D1-S03/learner/	starting files
outputs/	what you create
.virtual-env-folder/	the course Python

CHECK

In the Explorer panel you see `case_pack`, `sessions` and `outputs` at the top level.

WATCH OUT

Opening the parent folder or a subfolder is the most common Day 1 problem: VS Code then cannot find the course Python or the data.

Also: open **Microsoft 365 Copilot Chat** in a browser tab and sign in.

Step 2: Two Python settings that can disagree (5 min)

	Interpreter	Notebook kernel
Set in	Ctrl+Shift+P (Mac: Cmd+Shift+P) → Python: Select Interpreter	Picker top right of an open notebook
Used by	<code>.py</code> scripts (D1-S03, S06–S08)	<code>.ipynb</code> notebooks (D1-S04)
Choose	The course <code>.virtual-env-folder</code>	The course <code>.virtual-env-folder</code>

CHECK

The path ends in `.virtual-env-folder\Scripts\python.exe` (Windows) or `.virtual-env-folder/bin/python` (macOS).

WATCH OUT

VS Code tracks these **separately**. Fixing one does not fix the other. Not listed? Choose **Enter interpreter path...**

Step 3: Run the readiness check (7 min)

In the VS Code **terminal** (the command window at the bottom of VS Code; open it with **View → Terminal**):

```
.virtual-env-folder\Scripts\python.exe case_pack\environment\preflight\smoke_test.py      (Windows)
.virtual-env-folder/bin/python case_pack/environment/preflight/smoke_test.py           (macOS)
```

What it checks

Check	Question
PF-01	Is this the course Python (3.12)?
PF-02	Am I in the course folder?
PF-03	Are pandas, openpyxl, matplotlib... installed?
PF-04	Can I find the data?
PF-05	Can I write to <code>outputs/</code> ?

Read the evidence, not the headline

```
RESULT: READY. All checks passed. You can
continue to D1-S03.
...
working_dir      : ...your course folder
in_project_venv : True      <- the evidence
```

A run can import everything and still be the **wrong** Python. `in_project_venv : True` proves it isn't.

Your working directory decides where files are found and saved

A **path** is a folder route written as text. Read it like File Explorer:

```
case_pack/data/clean/account_mapping.csv
```

Open `case_pack` > `data` > `clean` > the file. `/` and `\` both separate folders.

Paths here are **relative**: they start from your **working directory** (the folder you are working in).

Working directory	Result
The course folder	✓ File found
Your Desktop	✗ <code>FileNotFoundError</code> , or output saved somewhere you won't find it

Four locations to know

What	Where
December N01 transactions	<code>transactions/</code> in the data folder
Account mapping	<code>account_mapping.csv</code> in the data folder
Your output	<code>outputs/<session>/</code>
How to start over	The Reset note in the guide, or <code>RESET.md</code>

Data folder: `case_pack/data/clean/`

Tip: in VS Code, right-click a file → **Copy Relative Path** instead of typing it.

Step 4: Four common problems and what to try first (3 min)

Symptom	Most likely cause	What to try first
RESULT: NOT READY, PF-01 fails	Wrong Python (<code>in_project_venv</code> : False)	Run the command exactly as shown, starting <code>.virtual-env-folder\Scripts\python.exe</code>
<code>ModuleNotFoundError</code>	Usually the wrong Python (PF-01)	Check <code>in_project_venv</code> before installing anything
<code>FileNotFoundError</code> / PF-02	Wrong working directory	Open the course folder itself; check <code>working_dir</code>
Permission / access denied	Organisational restriction	Retrying will not help: send the support block

WATCH OUT

Do **not** run `pip install` to fix a missing package on a managed machine. Check the interpreter first; ask second.

Step 5: Report your status (2 min)

EXERCISE

Tell the instructor **one** of:

1. **READY**: result READY **and** `in_project_venv` : True
2. **Support block**: copy everything between COPY FROM HERE and COPY TO HERE and send it
3. **Paired**: you work alongside a partner whose setup works, and still make the decisions

The support block contains no username, home path or credentials.

Habits for today: save with **Ctrl+S** before running · run from the course folder · extract ZIPs into the course folder

Next (09:45, D1-S03)

Eight rows
One checked number
S03

Three files
Nine reporting lines
S04

In a workbook
Python in Excel
S05

Controls
Exceptions and release
S06

Formatted pack
Template filled
S07

Whole month
Run, decide, hand over
S08

We use Copilot Chat to write code for **eight rows**, run it, and check whether the answer is **720.00**.

D1-S03 · 09:45

Working with Microsoft 365 Copilot

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

Today's first question

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

The question

"What was unit **N01**'s operating result for **December 2025**?"

We start with the smallest possible case: **eight rows** you can check with a pen.

What you will produce

- A **specification** anyone could follow
- AI-generated code that you **ran** yourself
- **One number** you can show is right
- One **correction** of faulty AI-generated code

The loop, step by step



Step	What you do	Where
Specify	Write the rules, the output and the number you expect	Specification template
Generate	Paste the specification; get code	Microsoft 365 Copilot Chat
Execute	Copy the code into a file; run it	VS Code
Inspect	Print the rows used; compare with your number	VS Code
Correct	Ask for the smallest change; re-run	Copilot Chat → VS Code
Document	Note the check and the correction	Your notes

KEY IDEA

Generating code is **one step of six**. The others are yours.

The data and the rules (predict first)

File: `case_pack/data/clean/tiny/transactions_2025-12_N01.csv`

ID	Account	Amount	Status
T001	0400	1000.00	posted
T002	0410	500.00	posted
T003	0400	-100.00	posted
T004	0500	-400.00	posted
T005	0500	-200.00	posted
T006	0500	+20.00	posted
T007	0600	-100.00	posted
T008	0400	999.00	cancelled

Rules supplied by the business

1. Revenue stored **+**, costs stored **-** → result = **sum**
2. Only **posted** rows count
3. A **+** on a cost line is a reversal: **keep the sign**
4. `account_code` is **text** (keep the leading zero)

PREDICT FIRST

Before any code exists, agree with your neighbour what N01's operating result should be, and why.

Vague request vs specification

Vague

"Write Python to total the transactions in my CSV."

The code will run without errors, but will probably **include the cancelled row**, and nothing warns you.

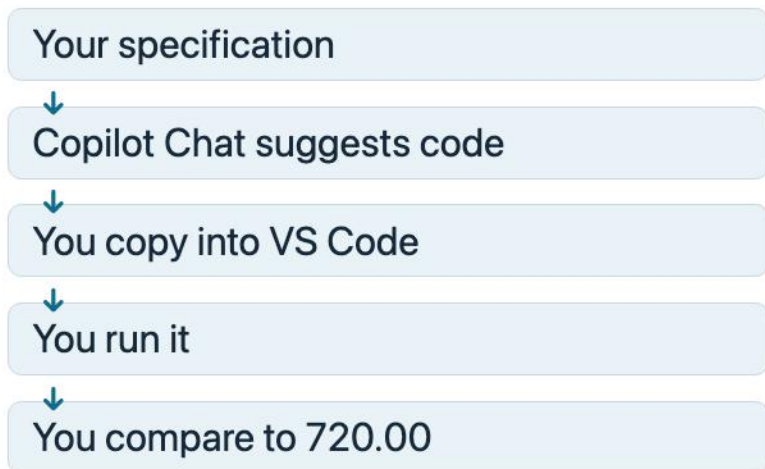
Specified (using the template headings)

- **TASK:** N01 operating result, December 2025
- **INPUT:** File path; columns; `account_code` is text
- **BUSINESS RULES:** Posted rows only; signs as stored; reversals keep their sign
- **OUTPUT:** One figure, 2 decimals, printed
- **CONSTRAINTS:** Use pandas; read and print only
- **ACCEPTANCE:** Expect **720.00**; avoid **1,719.00**

KEY IDEA

The acceptance example turns "looks fine" into a **test**.

Where things run, and your safety check



Copilot Chat only knows **what you paste**, and it does not run code on **your machine**. So **you** run it on the real files, and **you** check it.

WATCH OUT

Before you paste: Synthetic course data only. Never paste passwords or real customer data.

Before you run: Read the code. Does it **write, move or delete** files, or **connect to the internet**? If you didn't ask for it, don't run it.

Demonstration: one pass through the loop (9 min)

INSTRUCTOR DEMO

1. Paste the filled-in specification into Copilot Chat
2. Read the reply out loud: what does each line do? Does it write any files?
3. Save as `outputs/D1-S03/demo_total.py` and run it
4. Print the rows it used: is **T008** there?
5. Compare the printed number with the predictions on the board

CHECK

If the live answer is not 720.00, or produces an error, **that is useful**: we inspect the result rather than trust it.

Your turn: specify, generate, run, check (15 min)

EXERCISE

Guide section 3 · Individual

1. Fill in `sessions/D1-S03/learner/specification_template.md`
2. Paste it into Microsoft 365 Copilot Chat; **read** the code
3. Save it as `outputs/D1-S03/my_total.py` (right-click outputs → **New File**) and run it from the course folder:

```
.virtual-env-folder\Scripts\python.exe outputs\D1-S03\my_total.py      (Windows)
.virtual-env-folder/bin/python outputs/D1-S03/my_total.py              (macOS)
```

4. Compare the printed number with your prediction, and add a short comment in `my_total.py` noting the check you used

CHECK

A good check is concrete, e.g. "printed the rows used: 7, and T008 is not included".

Two ways AI-generated code goes wrong

Loud: the code stops with an error

```
KeyError: 'Status'
```

- Inconvenient, but **safe**: you know something is wrong
- Read the **last line** first, then the line in **your** file

Silent: the code prints a wrong number

```
Rows used: 8  
Operating result for N01, 2025-12: EUR 1,719.00
```

- Looks convincing, which makes it **risky**
- Only your **prediction** catches it

KEY IDEA

A loud failure costs minutes. A silent failure ends up in the management pack.

Your turn: diagnose and correct (10 min)

EXERCISE

Loud failure · Guide section 4A ·

`flawed_example_runtime.py`

1. Read the last line: which column?
2. `print(transactions.columns)` : what is it really called?
3. Ask Copilot for a **one-line** fix; run it

EXERCISE

Silent failure · Guide section 4B ·

`flawed_example_silent.py`

1. It prints **1,719.00** from **8** rows. Which rule was ignored?
2. Apply the rule; re-run
3. Confirm **720.00** from **7** rows

```
.virtual-env-folder\Scripts\python.exe sessions\D1-S03\learner\flawed_example_runtime.py  
.virtual-env-folder\Scripts\python.exe sessions\D1-S03\learner\flawed_example_silent.py
```

Edit the example files directly; to start again, extract them from the course ZIP · macOS: `.virtual-env-folder/bin/python` and `/`

A proposed fix vs a tested fix

A claim

"I've fixed it: the code now correctly filters posted rows."

An AI explanation, however confident, is only a claim.

Evidence

Rows used: 7

Operating result for N01, 2025-12: EUR 720.00

A re-run that matches your prediction is evidence.

KEY IDEA

Ask for the **smallest** change, run it, and check the explanation **against what actually ran**.

Explain, and what comes next

EXERCISE

In pairs · 5 min · Guide section 5

1. **One check** you ran, and what it showed
2. **One correction** you made, and how you know it worked

Optional: make costs positive **before** summing. Predict, run, and explain why **680.00** is wrong.

After the break (10:45, D1-S04)

The same loop at full scale:

3 unit files



362 rows



9 reporting lines

In VS Code, with a notebook, a mapping table and spot checks at every step.

D1-S04 · 10:45

The working method end to end

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

From eight rows to a whole month

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

The question

"What did **each unit** earn in December, and can you show me **where every number came from**?"

Why VS Code, not one workbook?

Several files, repeated every month, and we need a **trail** from each figure back to its source row.

3 unit CSVs + mapping



362 rows



9 reporting lines

	revenue	direct_cost	overhead
N01	?	?	?
N02	?	?	?
N03	?	?	?

Output: `outputs/D1-S04/actuals_by_line.csv`

The inputs: which files, and why not "all of them"

case_pack/data/clean/	
transactions/	
transactions_2025-10_N01.csv	not December
transactions_2025-11_N01.csv	not December
transactions_2025-12_N01.csv	✓ December
transactions_2026-01_N01.csv	not December
... (same for N02, N03)	
input_manifest_2025-12.json	names the 3 December files
account_mapping.csv	account code → report line
source_controls.csv	each unit's own totals

WATCH OUT

Loading "everything in the folder" pulls in **four months** and inflates every total, **without any error**.

KEY IDEA

The **manifest** fixes the list of files in advance. It is a business control, not a technical detail.

Reading a single row

transaction_id	period	entity	account_code	amount	currency	status
T000001	2025-12	N02	0500	-264.63	EUR	posted

The key

T000001 exists in **all three** units.
→ a row is identified by **entity + period + transaction_id**

Text, not a number

0500 read as a number becomes 500 and **no longer matches** the mapping.
→ read everything as text
(dtype=str)

Signs and status

Costs **-**, revenue **+**. Only **posted** counts. Reversals keep their sign.
→ same rules as the eight rows

Introducing the DataFrame (notebook section 1)

A **DataFrame** is a table in Python you can ask questions of.

```
rows.shape          # (362, 9)
rows.dtypes         # all text
rows.head()         # first 5 rows
rows["status"].value_counts()
# posted 354, cancelled 8
```

Four operations you will use today

Operation	pandas	Question it answers
Filter	<code>rows[rows["status"] == "posted"]</code>	Which rows count?
Merge	<code>posted.merge(mapping, ...)</code>	Which report line?
Group by	<code>.groupby([...])</code> <code>["amount"].sum()</code>	Total per unit and line?
Check	<code>len(...), .sum()</code>	Did anything change that shouldn't?

The supplied cell adds `source_file` and `source_row` to every row: the **trail**.

Predict before you run (notebook section 2)

PREDICT FIRST

Write these into the prediction cell **before** any transformation code:

1. How many rows are **posted**? (hint: `source_controls.csv`)
2. After joining the mapping, how many rows should there be?
3. What is **N01's operating result**?

`source_controls.csv`, December (the units' own records, independent of our code):

Entity	Raw rows	Posted rows	Posted amount
N01	119	117	29,862.50
N02	119	116	23,444.83
N03	124	121	14,023.53

How the mapping join works

DataFrame `posted` (354 rows)

entity	account_code	amount
N01	0400	855.70
N02	0500	-264.63

DataFrame `mapping` (one row per code)

account_code	report_line
0400	revenue
0500	direct_cost

DataFrame `joined` (still 354 rows)

entity	account_code	amount	report_line	_merge
N01	0400	855.70	revenue	both
N02	0500	-264.63	direct_cost	both

KEY IDEA

A join should **label** rows, never add rows or change the money. `_merge = "left_only"` would mean a code with **no** mapping.

Demonstration: filter → map → aggregate

INSTRUCTOR DEMO

```
posted = rows[(rows["status"] == "posted") & (rows["currency"] == "EUR")].copy()
posted["amount"] = posted["amount"].astype(float)

joined = posted.merge(mapping, on="account_code", how="left",
                      validate="many_to_one", indicator=True)

by_line = (joined.groupby(["period", "entity", "report_line"], as_index=False)["amount"].sum()
           .rename(columns={"amount": "actual_amount"}))
# then order rows revenue, direct_cost, overhead within each unit
```

Safeguard	Without it
<code>how="left" + indicator=True</code>	Unmapped rows silently disappear
<code>validate="many_to_one"</code>	A duplicated mapping silently multiplies rows
<code>.astype(float)</code> after filtering, no <code>abs()</code>	Reversals change sign and totals are wrong

Your turn: build it (notebook section 3, 30 min)

EXERCISE

Guide section 5 · Individual

1. Write a short specification (D1-S03 template): posted EUR rows · amount converted to a number · left join on text `account_code` with `validate` and `indicator` · signed totals by period/entity/report_line as `actual_amount` · business row order
2. Ask Copilot; paste into the **YOUR TURN** cell; **read before running** (file writes? inner join? `abs ( ) ?`)
3. Run it, then run the **spot checks** cell (next slide)

FILES

Notebook: `sessions/D1-S04/learner/staged_notebook.ipynb` · Variables to create: `posted`, `joined`, `by_line`

Spot checks at every step (notebook section 4)

CHECK

The spot-check cell should print **True** on every line:

Check	Expect
Rows after filter	354
Rows after join	354 (unchanged)
Sum of amount before vs after join	Identical
<code>_merge</code> values	All <code>both</code>
N01 operating result	29,862.50

If one prints False

False line	Look at
Rows after filter	The status and currency filter
Join added rows	Duplicated codes in the mapping
Money changed	The join, or <code>abs()</code>
Not all mapped	<code>joined[joined["_merge"] != "both"]</code>
N01 differs	The rows for N01 and their signs

Test a known fault: a duplicated mapping (notebook section 5)

PREDICT FIRST

case_pack/data/faulty/F03_duplicate_mapping/account_mapping.csv lists code **0500** twice (direct_cost **and** overhead).

What will `validate="many_to_one"` do?

EXERCISE

Guide section 6 · 15 min

1. Rerun your join with the faulty mapping
2. Remove `validate=` once and count the rows
3. Restore the clean mapping and **rerun section 3**

	Clean	F03, with validate	F03, without validate
Result	354 rows	MergeError	455 rows
Total	67,330.86	n/a	26,318.19
Error?	No	Yes (loud)	No (silent)

WATCH OUT

Don't delete the duplicate yourself. The **mapping owner** decides which row is right.

Save and compare with the answer (notebook section 6)

The compare cell writes your CSV and matches it against the checkpoint:

```
case_pack/data/clean/checkpoints/actuals_by_line_2025-12.csv
```

Entity	Revenue	Direct cost	Overhead
N01	68,197.95	-28,059.68	-10,275.77
N02	50,612.03	-19,556.86	-7,610.34
N03	38,117.91	-17,554.80	-6,539.58

CHECK

```
all 9 keys match: True
all amounts within 0.01: True
business row order: True
```

Unit results: N01 **29,862.50** · N02 **23,444.83** · N03 **14,023.53** · total **67,330.86**

Not finished? Use the checkpoint file this afternoon.
Nothing later depends on this notebook.

Explain, and what comes next

EXERCISE

In pairs · 5 min · Guide section 7

In business language:

1. How could a duplicated reference row change a plausible total?
2. Why do `source_file` and `source_row` matter when something is wrong?
3. Why keep the positive reversal instead of using `abs()`?

You now have

Nine checked reporting lines for December, with a trail to every source row.

After lunch (13:15, D1-S05)

Back to a **simpler tool**: the same nine lines are already in a workbook. Can Python **inside Excel** do the analysis and a chart?

D1-S05 · 13:15

Python in Excel

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

Back to a simpler tool

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

This morning

Three CSV files, repeated monthly, a trail to every row →

VS Code

This afternoon

The nine lines are **already in a workbook**, and the audience works in Excel → **Python in Excel**

KEY IDEA

Question: can we do the analysis and a chart **inside the workbook**, check it, and decide whether Python was even needed?

Two architectures, side by side

	Python in Excel	Local Python (VS Code)
Where code runs	Microsoft cloud container	Your machine
What it can read	Workbook data via <code>xl()</code> only	Any file you can open
Files / internet from code	No	Yes
Libraries	Fixed Anaconda set (pandas, matplotlib...)	What your environment installs
Result lives in	A cell: <code>=PY(...)</code>	Files you write
Who can rerun it	Anyone with the workbook and the feature	Anyone with the script and environment

WATCH OUT

Included with Microsoft 365 Business and Enterprise plans (Windows, Mac, web), but your organisation must have it **enabled**. Can you see **Formulas → Insert Python?**

The starting workbook

sessions/D1-

S05/learner/northbridge_slice.xlsx → sheet

Inputs → table **NorthbridgeLines**

period	entity	report_line	actual_amount
2025-12	N01	revenue	68,197.95
2025-12	N01	direct_cost	-28,059.68
2025-12	N01	overhead	-10,275.77
2025-12	N02	...	...
2025-12	N03	...	...

These are **this morning's nine checked lines** (the D1-S04 checkpoint).

EXERCISE

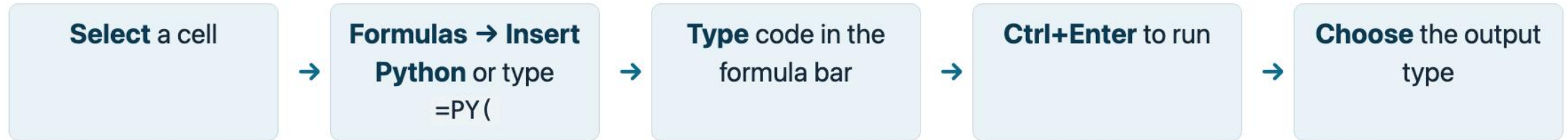
Guide section 1

1. **File** → **Save As** outputs/D1-S05/northbridge_exercise.xlsx
2. Never work on the original

PREDICT FIRST

Each unit's operating result = the sum of its three lines. What should N01 be?

Creating a Python cell



Output types

- **Python object** (card icon): Best for passing a DataFrame to the next Python cell
- **Excel values**: Fills the cells below, so Excel charts and formatting can use it

Calculation order

Python cells run **left to right, then top to bottom** (row-major).

A cell can only use variables from cells **above or to the left**.

Cell 1: from table to unit results

INSTRUCTOR DEMO

```
lines = xl("NorthbridgeLines[#All]", headers=True)
unit_result = lines.groupby("entity", as_index=False)["actual_amount"].sum()
unit_result = unit_result.rename(columns={"actual_amount": "operating_result"})
unit_result
```

Output type: **Excel values**

Code	Means
<code>xl("NorthbridgeLines[#All]", headers=True)</code>	The whole table, with the first row as column names
<code>groupby("entity")...sum()</code>	Adds up the three lines per unit
Last line <code>unit_result</code>	What the cell returns

CHECK

entity	operating_result
N01	29,862.50
N02	23,444.83
N03	14,023.53

Cell 2: a chart

INSTRUCTOR DEMO

In a cell **below** cell 1 (e.g. A6):

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.bar(unit_result["entity"], unit_result["operating_result"])
ax.set(title="Northbridge December operating result",
        ylabel="EUR, signed")
fig
```

By default the cell shows a **card icon** (an image object). Click the card to preview it, or right-click → **Display Plot over Cells** for a large, movable chart.

WATCH OUT

`NameError: unit_result` → the chart cell is **above or left of** cell 1. Move it.

Code for both cells: `sessions/D1-S05/learner/workbook_python.txt`

Your turn: analysis and chart (18 min)

EXERCISE

Guide sections 2 and 3 · In your exercise copy

1. On a new sheet, cell **A1**: enter **cell 1**, Ctrl+Enter, set output to **Excel values**
2. Cell **A6**: enter **cell 2** (chart), Ctrl+Enter
3. **Or**: Ask Microsoft 365 Copilot Chat to write the code from a short specification, paste it in, then **read** it
4. Check the three unit results against the table on slide 5

FILES

`outputs/D1-S05/northbridge_exercise.xlsx` · `sessions/D1-S05/learner/workbook_python.txt`

No access? Pair with someone who has it, and tell the instructor exactly what failed (e.g. “no Insert Python button”).

Change an input: is the result linked to the data? (7 min)

PREDICT FIRST

Add **EUR 100** to N01 revenue: 68,197.95 → **68,297.95**.

What changes in the table? In the chart?

EXERCISE

Guide section 4

1. Make the change; Python cells recalculate
2. Check the results
3. Restore 68,197.95 (copy it from the untouched original)
4. Save, close, reopen: results still there?

CHECK

Entity	Before	After
N01	29,862.50	29,962.50
N02	23,444.83	23,444.83
N03	14,023.53	14,023.53

Only N01 moves, by exactly +100. That proves the cell is **calculated from the data**, not a pasted number.

Was Python even needed?

For this sum: no

A **PivotTable** or **SUMIFS** does it in a minute.

Python in Excel is worth using when

The analysis needs **pandas, statistics or specialised charts**, but the data and audience stay in the workbook.

VS Code is worth using when

You need **several files, repeated runs, controls, or a formatted output file**.

Next (14:00, D1-S06)

Before anyone uses these figures: **controls**. How do we *know* no file is missing, nothing is duplicated and every total reconciles?

D1-S06 · 14:00

Data quality, controls and reconciliation

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

Can we release December's figures?

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

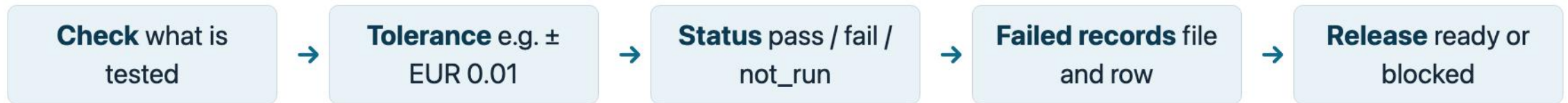
This morning you **spot-checked** one result by eye.

A **control** is that check written once, run **every month**, with a **recorded outcome**.

A total that *looks* right can't tell you a file is missing or a row was counted twice. **Controls can.**

What you will produce

- Two control functions you write yourself
- `control_summary.csv`: every check and its status
- `exceptions.csv`: failed records with **file and row**
- A release decision: **ready** or **blocked**



Northbridge's nine controls

ID	Check	Catches	Today
C01	Expected files present	A unit's export never arrived	Supplied
C02	Row counts vs unit control totals	Extra or missing rows	Supplied
C03	Duplicate business key	The same transaction twice	You write it
C04	Mapping has one row per code	A duplicated reference row (F03 this morning)	Supplied
C05	Every posted code is mapped	A new account nobody mapped	Supplied
C06	Required values present	Blank amount, invalid status	Supplied
C07	Currency is EUR	Mixed currencies	Supplied
C08	Posted amount reconciles, ± EUR 0.01	Money differs from the unit's total	You write the comparison
C09	Movement vs last month (review only)	Large swings to explain	Supplied

C01–C08 are **critical**: any fail blocks release. C09 is a **review warning**: someone explains it; it doesn't block.

Three outcomes, not two

Status	Means	Release?
pass	Check ran; nothing wrong	Yes
fail	Check ran; found problems	No (blocked)
not_run	Check could not run	No (blocked)

KEY IDEA

A not_run status is never a pass. “We didn’t look” is not “nothing is wrong”.

Example: the reconciliation can’t run because a file is missing. Should the pack go out? **No.**

Setup and demonstration: run the supplied controls

EXERCISE

Setup (everyone) · Guide section 2

1. Get **D1-S06 component_handout.zip** from the instructor
2. Extract it **into the course folder** (in Windows Extract All, remove the suggested component_handout folder)
3. Check outputs/handoffs/D1-S06/report_components.py exists

INSTRUCTOR DEMO

check_id	status	affected_count	
C01	pass	0	
C02	pass	0	
C03	not_run	0	<- your task 1
C04	pass	0	
...			
C08	not_run	0	<- your task 2
C09	not_run	0	<- needs C01-C08
Release:	blocked		

```
.virtual-env-folder\Scripts\python.exe sessions\D1-S06\learner\run_controls.py
```

Writes `outputs/D1-S06/control_summary.csv` and `exceptions.csv`.

Task 1: find duplicates, keep the evidence

```
KEY = ["entity", "period", "transaction_id"]

def duplicate_rows(rows):
    """Return EVERY occurrence of a duplicated
    full key, with source_file and source_row."""
```

PREDICT FIRST

Which rows below should it return?

Entity	Transaction ID	Source row
N01	T000001	1
N02	T000001	1
N01	T000042	42
N01	T000042	89

CHECK

Rows **42 and 89** only. T000001 in N01 and N02 are **different** transactions.

WATCH OUT

Return **all** occurrences, not just “the extra one”, and never delete duplicates. The unit decides which one is genuine.

Task 2: reconciliation within a tolerance

```
def reconciliation_status(observed, expected):  
    """'pass' if |observed - expected| <= 0.01  
    (inclusive), otherwise 'fail'."""
```

C08 calls this for each unit: our posted total (**observed**) vs the unit's own total (**expected**).

WATCH OUT

In Python, `abs(14023.54 - 14023.53) <= 0.01` is **False** (0.01000000000000218). Compare **cents** (`round(x * 100)`) or use `Decimal`.

PREDICT FIRST

Control total **14,023.53** (N03):

Observed	Difference	Status?
14,023.54	+0.01	
14,023.55	+0.02	
14,023.52	-0.01	
14,023.51	-0.02	

Your turn: write, test, run (20 min)

EXERCISE

Guide sections 3 to 5 · Edit `sessions/D1-S06/learner/control_stubs.py`

1. Decide what you expect for slides 5–6
2. Complete `duplicate_rows` and `reconciliation_status` (Copilot is fine; **read** the code)
3. **Self-test:** `.virtual-env-folder\Scripts\python.exe sessions\D1-S06\learner\control_stubs.py` → every line ok
4. **Clean run** into `outputs\D1-S06\clean` → all pass, **ready**
5. **Faulty runs**, F02 and F06, each into its own folder

The exact commands to copy are in the participant guide, section 5.

Read the exception report

F02: one duplicated row triggers three controls

Check	Where	Issue
C03	N01 row 42	Duplicate key T000042
C03	N01 row 89	Duplicate key T000042
C02	N01	Raw rows 120 vs control 119
C08	N01	Posted amount differs by –660.87

F06: the posted amount doesn't reconcile

Check	Where	Issue
C08	N01	Posted 31,362.50 vs control 29,862.50 (+1,500.00)

KEY IDEA

Each exception has an **action note**, e.g. “Ask the unit which occurrence is genuine; never drop duplicates to make counts match.” That is what you send to the unit.

Explain, and what comes next

EXERCISE

2 min · Guide section 7

1. Why is a failed critical check different from a C09 warning?
2. Why can `not_run` never count as a pass?
3. What would you send N01 about F02?

You now have

Controls that produce a **release decision** and **traceable exceptions**.

After the break (15:00, D1-S07)

Put the checked figures, **or** the blocked diagnosis, into the **formatted management template**.

D1-S07 · 15:00

Reporting automation

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

From checked figures to the management pack

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

The question

Checked figures are no use until they are in **the pack management already reads**.

Can we fill the existing template **automatically**, keep its formatting, save a **new dated file**, and never touch the master?

What you will produce

- `outputs/D1-S07/run-01/Northbridge_Management_Pack_2025-12_r01.xlsx` (**ready**)
- `outputs/D1-S07/run-02/..._r01_BLOCKED.xlsx` (**blocked**)
- `outputs/D1-S07/handover.md`

KEY IDEA

This is the **December pack** from D1-S01. You now have every ingredient: figures (D1-S04) and controls (D1-S06).

The template: five sheets and what goes in each

Sheet	Receives	Sign
Summary	Period, status, one row per unit	Costs shown positive
ByLine	The 9 reporting lines and budget	Signed , as stored
Controls	The control summary	n/a
Exceptions	Failed records	n/a
ReadMe	Inputs, run details, how to rerun	n/a

Summary sheet layout

```
C3   Period           2025-12
C4   Release status   ready
row 8 (B-I) Unit | Unit name | Revenue |
      Direct cost | Overhead | Operating result |
      Budget operating result | Favourable variance
rows 9-11 N01, N02, N03
row 12      Total
C14  =ROUND(G12-(D12-E12-F12),2)
```

KEY IDEA

Stored vs displayed (from the primer): the data stores direct cost as -28,059.68; the Summary **displays** 28,059.68. Same number, different label.

Predict: what does Python see in cell C14?

PREDICT FIRST

C14 contains a **formula**. After Python saves the file:

1. What does Python see if it reads C14 back?
2. What does Excel show when you open the file?

CHECK

1. Python (openpyxl) sees the **formula text**:
`'=ROUND ( G12 - ( D12 - E12 - F12 ) , 2 ) '`
2. Excel **calculates** it on opening: **0.00**

KEY IDEA

The openpyxl library **writes** files; it does not **calculate** them. “The file saved” is not “the numbers are right”. Numbers are checked on the data; Excel shows the formula result.

The supplied function that writes the workbook

```
report.write_workbook(output_folder, inputs, release_status,  
                      control_summary, exceptions, summary)
```

What it already does

- Opens the **master** and saves a **new** file named `Northbridge_Management_Pack_<period>_r01.xlsx`
- **Refuses** to overwrite the master or an existing output
- Writes all five sheets, keeping the formatting

When `release_status == "blocked"`

- **No figures** on Summary and ByLine
- Controls and Exceptions written **in full**
- `_BLOCKED` added to the filename

EXERCISE

Setup (everyone) · Guide section 1: extract **D1-S07 component_handout.zip** into the course folder (in Windows Extract All, remove the suggested `component_handout` folder).

Demonstration: write, open, rerun

INSTRUCTOR DEMO

```
inputs, controls, exceptions = read_control_run(
    "clean_2025-12")
path = report.write_workbook(
    Path("outputs/D1-S07/run-01"),
    inputs, "ready",
    controls, exceptions, summary)
print(path)
```

Then run the **same** call again:

```
FileExistsError: ..._r01.xlsx
already exists; rerun with
overwrite=True to replace it
```

That refusal is the protection working.

CHECK

Summary sheet in Excel (costs displayed positive) · C14 shows **0.00**

Unit	Revenue	Direct cost	Overhead	Operating result
N01	68,197.95	28,059.68	10,275.77	29,862.50
N02	50,612.03	19,556.86	7,610.34	23,444.83
N03	38,117.91	17,554.80	6,539.58	14,023.53
Total	156,927.89	65,171.34	24,425.69	67,330.86

Your turn: fill and save the template (18 min)

EXERCISE

Guide section 4 · Edit `sessions/D1-S07/learner/writer_scaffold.py`

The starter prepares `summary` and `read_control_run(name)`; three **settings** at the top choose the run.

1. Complete `populate_and_save(inputs, controls, exceptions, release, output_folder)` so it calls `report.write_workbook` and returns the path (ask Copilot; **read** each argument it passes)
2. Run the script with the default settings: `clean_2025-12 · ready · run-01`
3. Open the file: check Summary against slide 5, C14 = 0.00, formatting intact, ByLine signed
4. Run the script again without changes: confirm it **refuses**

WATCH OUT

Common AI mistakes: saving to the **template path** · hard-coding `"ready"` · building a new workbook with `Workbook()` (formatting lost) · writing costs negative on Summary

Produce the blocked pack (8 min)

EXERCISE

Guide section 5

Change the three settings, then run again:

```
CONTROL_RUN = "F03_duplicate_mapping"  
RELEASE = "blocked"  
OUTPUT_FOLDER = Path("outputs/D1-S07/run-02")
```

F03 is this morning's duplicated mapping: code **0500** mapped twice.

CHECK

- Filename ends `_BLOCKED.xlsx`
- Summary: *"Figures withheld: release blocked."*
- ByLine: **no figures** (a short blocked note)
- Controls: C04 **fail**
- Exceptions: `account_mapping.csv` rows **4 and 8**

KEY IDEA

A blocked pack is still useful: **the control results and exceptions are what you hand over**. A file that saves is **not** an approved file.

Handover, and what comes next

EXERCISE

2 min · Guide section 6 · Write `outputs/D1-S07/handover.md`, one line each:

**Inputs · Period · Output file · Release status ·
How to rerun**

- Why is **preserving** a formula different from **calculating** it?
- Why is “it saved” not numerical proof?

Next (15:45, D1-S08)

Everything together on a **new month**:

January files



Controls



Decide



Pack



Handover

January's data contains a problem that you will need to find.

D1-S08 · 15:45

Guided end-to-end exercise

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

A new month arrives

Eight rows

One checked number
S03

Three files

Nine reporting lines
S04

In a workbook

Python in Excel
S05

Controls

Exceptions and release
S06

Formatted pack

Template filled
S07

Whole month

Run, decide, hand over
S08

The brief

January's three unit exports are in. Management wants the pack.

Your job: run the whole process, decide whether the figures can be released, and write a handover note a colleague could act on.

What you will produce

- `outputs/D1-S08/attempt-01/`: the first run and its decision
- `outputs/D1-S08/attempt-02/`: the corrected run (if justified)
- `outputs/D1-S08/handover.md`

EXERCISE

Setup · Guide section 1: extract **D1-S08 component_handout.zip** into the course folder (remove the suggested component_handout folder in Extract All), then run `.virtual-env-folder\Scripts\python.exe sessions\D1-S08\learner\monthly_pack.py` → it prints Inputs ready and Result: None

The whole day in one function

```
run_month(input_folder, period, output_folder, mapping_file=None)
```

```
1. inputs = load_inputs(folder, period, mapping_file=...)    S04
2. controls, exceptions, release = run_controls(inputs)      S06
3. write control_summary.csv + exceptions.csv    (always)
4. if release is not "blocked":
    actuals = build_actuals(...)                      S04
    summary = build_summary(...)
    write actuals_by_line.csv
else:
    summary = None
5. write_workbook(output_folder, inputs, release,           S07
                  controls, exceptions, summary)
6. return release, workbook_path
```

KEY IDEA

Controls first: figures are only built **after** the controls allow release.

KEY IDEA

Parameters, not edits: folder, period and mapping file are arguments, so the same code runs next month.

All functions come from `components` (already imported in the starter).

Write the specification before asking Copilot

EXERCISE

Guide section 2 · 10 min · Specification (D1-S03 template)

- **Inputs:** Folder, period 2026-01, optional `mapping_file`
- **Steps:** The six steps on slide 2, in order
- **Outputs:** Control CSVs always; `actuals_by_line.csv` only if released; workbook; a new folder per attempt
- **Returns:** (`release_status`, `workbook_path`)
- **Must not:** Edit the raw exports · hard-code a status · delete rows that cause problems

PREDICT FIRST

Which controls must pass before January's figures can be released?

What would you expect the workbook to be called if they don't?

Build and run (35 min)

EXERCISE

Guide section 3 · Edit `sessions/D1-S08/learner/monthly_pack.py`

1. Ask Copilot to implement `run_month(...)` from your specification
2. **Read it:** does it build figures **before** checking release? Does it hard-code "ready" ?
3. Run the script (setting `ATTEMPT = "attempt-01"`) → `outputs/D1-S08/attempt-01`
4. Open `control_summary.csv` and `exceptions.csv`
5. Answer in one sentence: **may management use these figures, and why?**

WATCH OUT

If your code produced a **released** pack from these inputs, your gate is wrong. A **blocked** result here is correct.

What the controls found

```
C01-C04 pass
C05 unmapped_accounts fail
C06-C08 pass
C09 movement_review not_run
Release: blocked
```

Northbridge_Management_Pack_2026-01_r01_BLOCKED.xlsx

Source file	Row	Account	Amount
transactions_2026-01_N02.csv	5	0430	1,705.48
transactions_2026-01_N02.csv	19	0430	3,632.11
... 6 rows in total, all N02			

Rows are data rows (editor line = row + 1). The action note in the file says, in short: ask finance for the mapping, and don't edit the export or drop the amount.

KEY IDEA

A new account code nobody has mapped. Dropping those rows would **under-report N02 revenue**, silently.

Finance replies: apply the correction

Finance sends **account_mapping_updated.csv** (in the `late_delivery/` subfolder of the input folder). It adds account **0430** → **revenue** (Workshop revenue) to the mapping.

EXERCISE

Guide section 4 Change the two settings at the top of `monthly_pack.py`, then run it again:

```
ATTEMPT = "attempt-02"
MAPPING_FILE = Path("case_pack/data/faulty/"
    "F10_january_unmapped_account/late_delivery/"
    "account_mapping_updated.csv")
```

WATCH OUT

- **Do not** edit the exports
- **New** folder: attempt-01's blocked diagnosis stays as evidence
- Record **which reference changed, who sent it, and why**

Validate the release (15 min)

EXERCISE

Guide section 5

1. Expect ready and a workbook **without** `_BLOCKED`
2. Run the comparison code in the guide against `case_pack/data/clean/checkpoints/actuals_by_line_2026-01.csv`
3. In Excel: Summary formatting intact, C14 = **0.00**

CHECK

```
all 9 keys: True  
within 0.01: True
```

The checkpoint was produced **independently** of the faulty folder, which is what makes it a real check.

No Excel? Record “C14 not checked in Excel” rather than assuming it.

The handover note

EXERCISE

Guide section 6 · outputs/D1-S08/handover.md

1. Purpose and period
2. Input folder and the **reference correction** (what, who, why)
3. Exact command to rerun
4. Controls and release decision for **both** attempts
5. Output locations
6. Known limitations

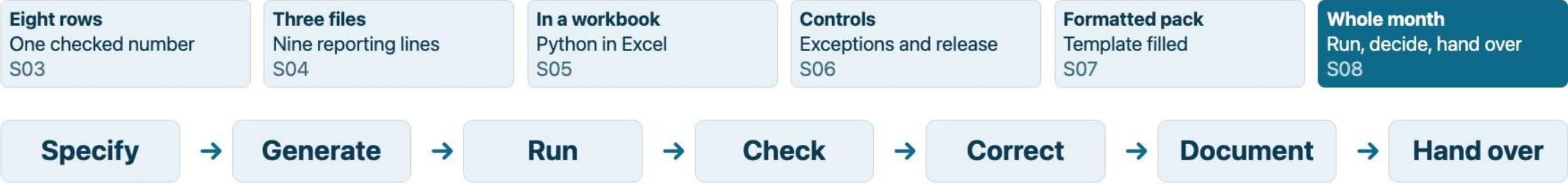
Peer check (5 min · Guide section 7)

Swap notes with a partner. Without asking you:

- Could they rerun it?
- Do they know what “blocked” meant and what changed?

Then explain **one prediction, one change, one check** in your own words.

Day 1 summary



What you did today

- Confirmed **720.00** on eight rows
- Combined three files into **9 checked lines**
- Analysed them **inside Excel**
- Wrote **controls** with a release decision
- Filled the **formatted pack**, ready or blocked
- Ran **January**, found a problem, and fixed it without editing the source data

Days 3–4: your project

Track	
Monthly reporting pack	Multi-file reconciliation
Audit exceptions and data validation	External data refresh with controls
Budget or forecast scenarios	Management dashboard

Every project includes: **tool choice rationale** · **evidence of checks** · **handover note**